

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192524098
Name	Divyasree.M

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 10

Total Marks: 25

Code of Conduct

I Divyasree.M/192524098 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Divyasree. M*

Assessment Tasks

Mock Interview

Answer the following interview-oriented questions clearly and concisely. Support your responses with architecture-level reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?
8. What are the key failure points in a Hadoop cluster and how would you handle them?
9. How would you design a secure data ingestion pipeline for a large organization?
10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mock Interview Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Knowledge	25%	Shows deep and accurate technical understanding	Good technical understanding	Basic understanding	Weak understanding
Problem Solving	20%	Responds with strong architecture-level reasoning	Reasonable reasoning	Basic reasoning	Weak reasoning
Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication
Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

Q1. Explain the difference between HDFS and traditional file systems.

HDFS (Hadoop Distributed File System) is purpose-built to store very large datasets reliably across clusters of commodity hardware, whereas traditional file systems (like NTFS or ext4) are designed to manage data on a single machine's local disk. The architectural differences drive very different behaviour:

- **Storage model:** Traditional file systems store files on one disk/node. HDFS splits files into large blocks (128 MB or 256 MB by default) and spreads them across many DataNodes in a cluster, so a single file can be larger than any individual disk.
- **Replication vs. RAID:** Traditional systems rely on RAID or backups for fault tolerance. HDFS natively replicates each block (default factor of 3) across different nodes/racks, so node failure does not mean data loss.
- **Access pattern:** Traditional file systems support general-purpose random read/write access. HDFS follows a write-once, read-many-times model optimized for large sequential scans (batch analytics), not low-latency random writes.
- **Metadata management:** A traditional file system keeps metadata on the same disk as the data. HDFS separates the two: a NameNode holds all metadata (namespace, block locations) in memory, while DataNodes hold the actual block data — enabling horizontal scaling.
- **Scale and hardware:** Traditional file systems assume reliable, expensive hardware. HDFS assumes commodity hardware will fail regularly and is engineered around that assumption, scaling to petabytes across thousands of nodes.

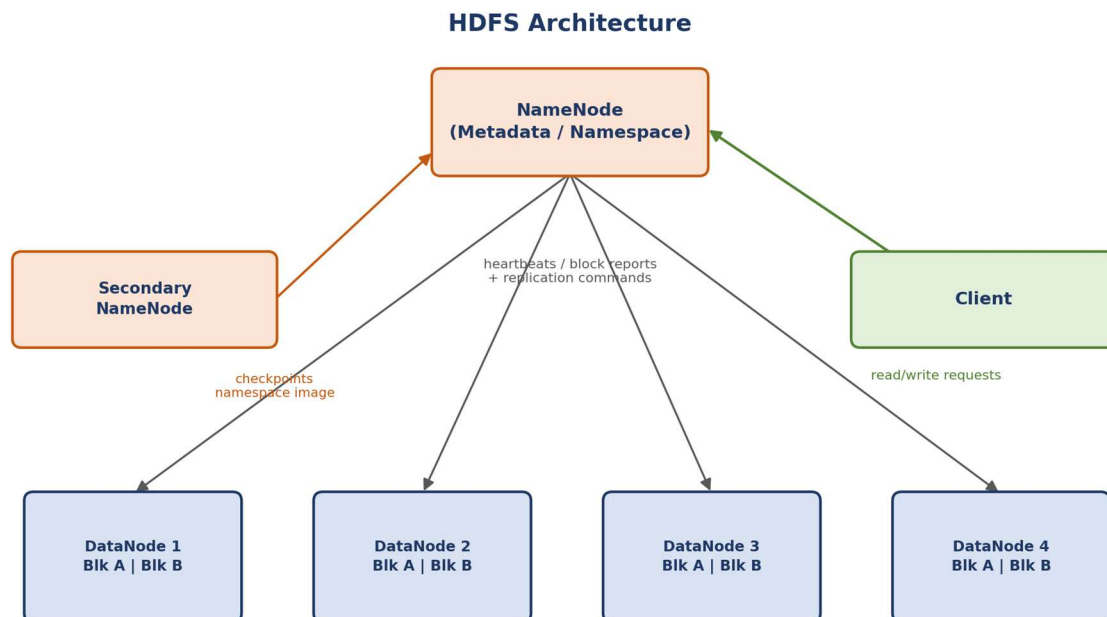


Figure 1: HDFS master-slave architecture — NameNode manages metadata while DataNodes store replicated blocks.

In short: a traditional file system optimizes for single-machine reliability and general-purpose access, while HDFS optimizes for distributed scale, fault tolerance, and high-throughput batch processing.

Q2. Why is replication important in distributed storage systems?

In a distributed system built on commodity hardware, node and disk failures are not exceptions — they are expected, routine events at scale. Replication is the primary mechanism that turns an inherently unreliable collection of machines into a reliable storage service.

- **Fault tolerance:** If a DataNode holding a block fails, HDFS still has other replicas of that block on different nodes, so no data is lost and the cluster keeps operating without manual intervention.
- **High availability:** Because data isn't dependent on a single node, jobs can keep reading and writing even during hardware failures, maintenance, or rolling upgrades.
- **Data locality and performance:** Having multiple copies across different nodes lets the scheduler place computation close to a copy of the data (data locality), reducing network I/O and speeding up MapReduce/Spark jobs.
- **Load distribution:** Popular blocks can be read from any of their replicas in parallel, spreading read load across the cluster instead of bottlenecking on one node.
- **Rack awareness:** HDFS's default replica placement policy (one replica on the local rack, others on a different rack) protects against an entire rack or switch failure, not just a single disk.

The trade-off is storage overhead — three-way replication triples raw storage cost — which is why some systems use erasure coding as a more storage-efficient alternative once data becomes "cold," trading some CPU and recovery-time cost for lower storage overhead.

Q3. How does MapReduce divide work between map and reduce phases?

MapReduce splits a large computation into two well-defined phases connected by a shuffle-and-sort step, allowing work to be distributed and executed in parallel across the cluster.

- **Input splitting:** The input dataset (typically residing in HDFS) is divided into fixed-size input splits, each roughly aligned to one HDFS block, so each split can be processed on the node where the data already lives.
- **Map phase:** A Map task is launched per input split. Each mapper reads its split as key-value pairs, applies the user-defined `map()` function independently and in parallel with all other mappers, and emits intermediate key-value pairs. Mappers do not communicate with each other.
- **Shuffle and sort:** The framework automatically partitions, sorts, and transfers the mapper output across the network so that all values sharing the same key end up together at the same reducer. This is the most network- and I/O-intensive step in the job.
- **Reduce phase:** Each Reducer receives one or more complete key groups, applies the user-defined `reduce()` function to aggregate/summarize the values for each key, and writes the final output back to HDFS.
- **Combiner (optional):** A local, mini-reduce step can run right after the map phase on the same node to pre-aggregate data before the shuffle, cutting down the volume of data sent across the network.

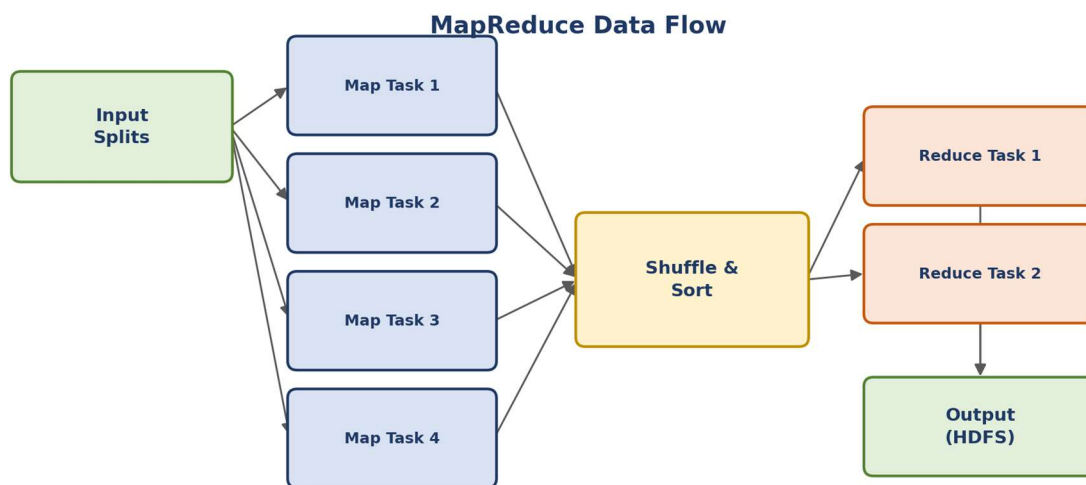


Figure 2: MapReduce data flow — parallel Map tasks feed a shuffle/sort stage that groups keys for the Reduce tasks.

This division lets MapReduce scale near-linearly: adding more nodes simply adds more mappers and reducers that run concurrently, while the framework — not the developer — handles parallelization, fault tolerance (re-running failed tasks), and data movement.

Q4. What role does YARN play in large-scale Hadoop processing?

YARN (Yet Another Resource Negotiator) is Hadoop's cluster resource management and job scheduling layer. It decouples resource management from the processing model, which is what allows Hadoop clusters to run MapReduce, Spark, Tez, and other engines side by side on the same shared infrastructure.

- **ResourceManager (RM):** The cluster-wide master that tracks available resources (CPU, memory) on every node and arbitrates them between competing applications via a pluggable Scheduler.
- **NodeManager (NM):** An agent running on every worker node that monitors local resource usage, launches and monitors Containers, and reports node health back to the ResourceManager.
- **ApplicationMaster (AM):** A per-job, per-application coordinator (itself running in a container) that negotiates resources from the RM and works with NodeManagers to execute and monitor the individual tasks of that specific job.
- **Container:** The unit of resource allocation — a bundle of CPU and memory on a specific node — inside which actual tasks (map tasks, reduce tasks, Spark executors, etc.) run.

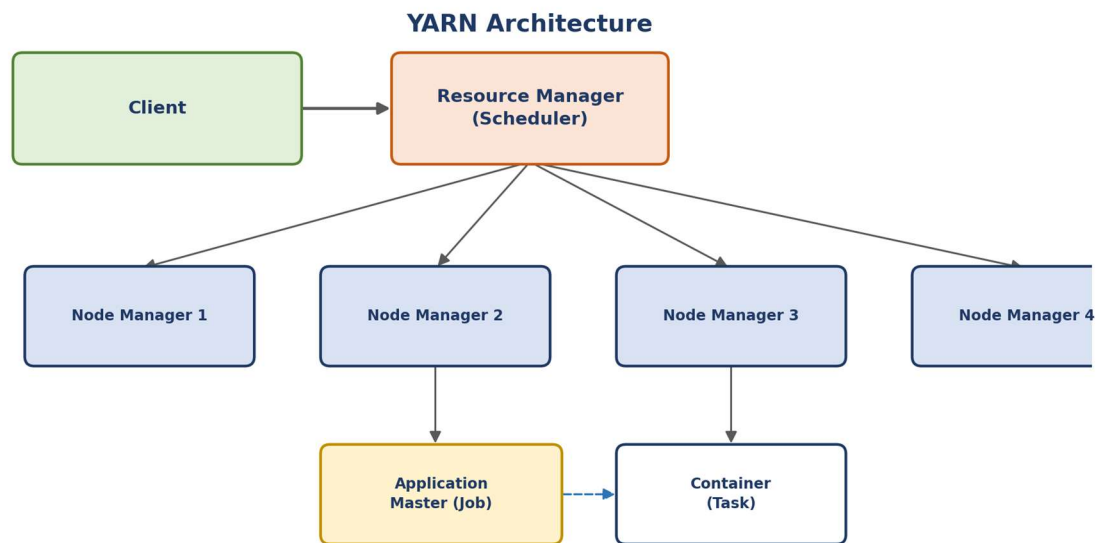


Figure 3: YARN separates cluster-wide resource negotiation (RM/NM) from per-job execution (ApplicationMaster/Containers).

Before YARN, Hadoop 1.x's JobTracker handled both resource management and job scheduling, which limited the cluster to MapReduce only and became a scalability bottleneck. YARN's separation of concerns lets the cluster scale to thousands of nodes and support multiple processing frameworks concurrently, improving both hardware utilization and job throughput.

Q5. Differentiate NAS and SAN in an enterprise data environment.

NAS and SAN are both networked storage architectures, but they operate at different levels of abstraction and suit different enterprise workloads.

Aspect	NAS (Network Attached Storage)	SAN (Storage Area Network)
Access level	File-level (NFS, SMB/CIFS)	Block-level (raw disk blocks)
Appears to client as	A shared network drive/folder	A locally attached disk
Typical network	Standard Ethernet / TCP-IP	Fibre Channel or iSCSI (dedicated fabric)
Best for	File sharing, home directories, unstructured data	Databases, transactional apps, high-IOPS workloads
Performance	Good; limited by general network traffic	Very high; low latency on a dedicated fabric
Management complexity	Simple to set up and administer	More complex; needs SAN-specific expertise
Relative cost	Lower cost, commodity hardware	Higher cost, specialized switches/HBAs

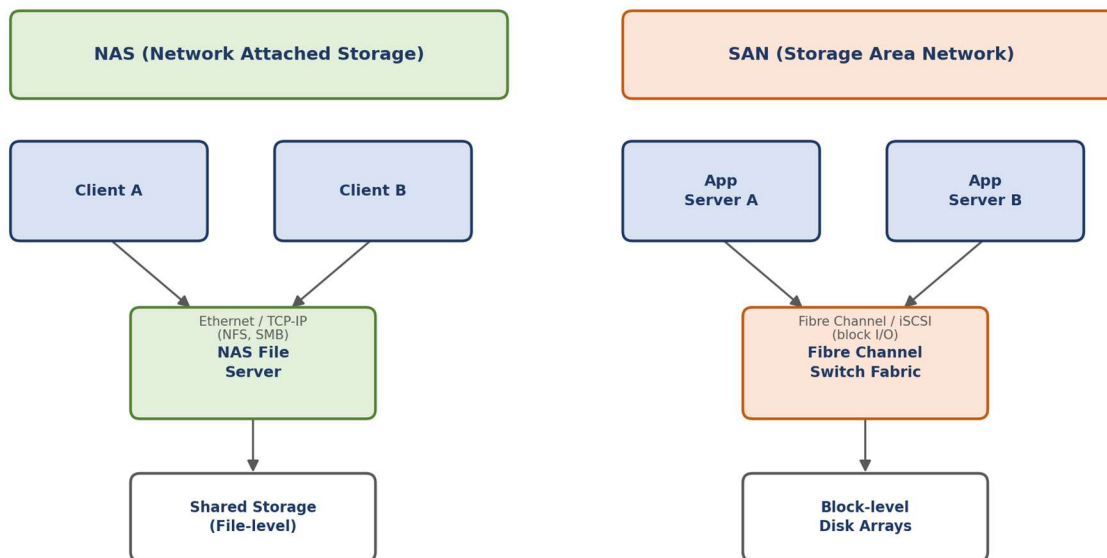


Figure 4: NAS provides file-level access over a standard network; SAN provides block-level access over a dedicated storage fabric.

In practice, large enterprises often use both: SAN for latency-sensitive transactional databases and virtualization storage, and NAS for general file shares, backups, and collaborative access — with big data platforms like Hadoop typically favoring commodity, directly-attached or NAS-style storage rather than expensive SAN, since HDFS already provides its own replication-based reliability.

Q6. What is the purpose of ETL in a big data pipeline?

ETL (Extract, Transform, Load) is the process that moves raw data from disparate source systems into a clean, structured, and analysis-ready form in a target data warehouse or data lake. Its purpose in a big data pipeline is to make heterogeneous, messy data usable and trustworthy.

- **Extract:** Data is pulled from varied sources — operational databases, application logs, third-party APIs, flat files, streaming systems — each with its own format and schema.
- **Transform:** Extracted data is cleaned (removing duplicates/errors), standardized (common formats, units, types), enriched (joined with reference/master data), and restructured (aggregations, business rules applied) so it conforms to the target schema.
- **Load:** The transformed data is written into the destination system — a data warehouse, HDFS/data lake, or analytical database — ready for BI tools, dashboards, or downstream ML pipelines.

ETL Pipeline (Extract - Transform - Load)



Figure 5: The ETL pipeline consolidates heterogeneous sources into a consistent, query-ready store.

Beyond just moving data, ETL enforces data quality and consistency, ensures data from different systems can be joined and compared meaningfully, and decouples analytics workloads from live operational systems so reporting doesn't impact production performance. In modern architectures this is sometimes reordered to ELT (load raw data first, transform inside the warehouse/lake using its compute power), but the underlying purpose — turning raw data into reliable, analysis-ready data — stays the same.

Q7. How does Apache NiFi help in data integration workflows?

Apache NiFi is a visual, flow-based tool for automating the movement, routing, and transformation of data between systems in real time. It is particularly valuable for building and operating data ingestion pipelines without writing large amounts of custom integration code.

- **Flow-based programming (FBP):** Pipelines are built as a directed graph of Processors (each doing one job — fetch, route, transform, publish) connected by Connections that carry FlowFiles (the unit of data plus its attributes).
- **Guaranteed delivery & provenance:** Every FlowFile is tracked through the FlowFile Repository, and NiFi maintains a detailed data provenance trail, so operators can trace exactly what happened to any piece of data and replay it if needed.
- **Back-pressure and prioritization:** Built-in back-pressure prevents a slow downstream system from being overwhelmed, and queues can be prioritized, which is critical when integrating systems with very different throughput characteristics.
- **Broad connector library:** Out-of-the-box processors connect to files, databases, REST APIs, Kafka, HDFS, cloud storage, and more, reducing the custom glue code needed for integration.
- **Web-based visual design and monitoring:** Flows are built and monitored through a browser UI in near real time, making it easier for both engineers and operators to understand, debug, and adjust running pipelines without redeploying code.

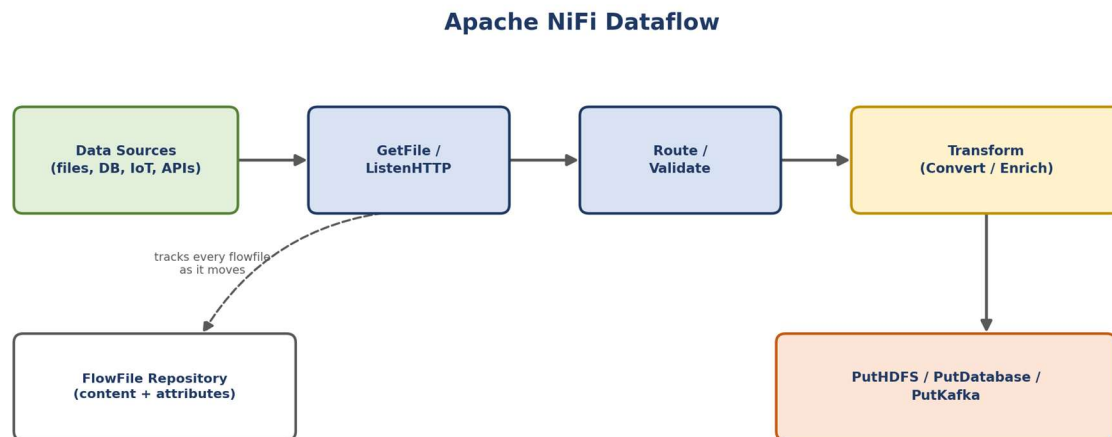


Figure 6: A typical NiFi flow — ingest, validate/route, transform, and deliver, with every FlowFile tracked end-to-end.

In a big data architecture, NiFi commonly sits at the ingestion edge — pulling data from many operational and external sources, applying light-weight routing/transformation/validation, and reliably delivering it into HDFS, Kafka, or a database for further processing.

Q8. What are the key failure points in a Hadoop cluster and how would you handle them?

Hadoop clusters are built from many commodity machines, so failures are expected. Reliability comes from anticipating specific failure points and designing the cluster to detect and recover from them automatically wherever possible.

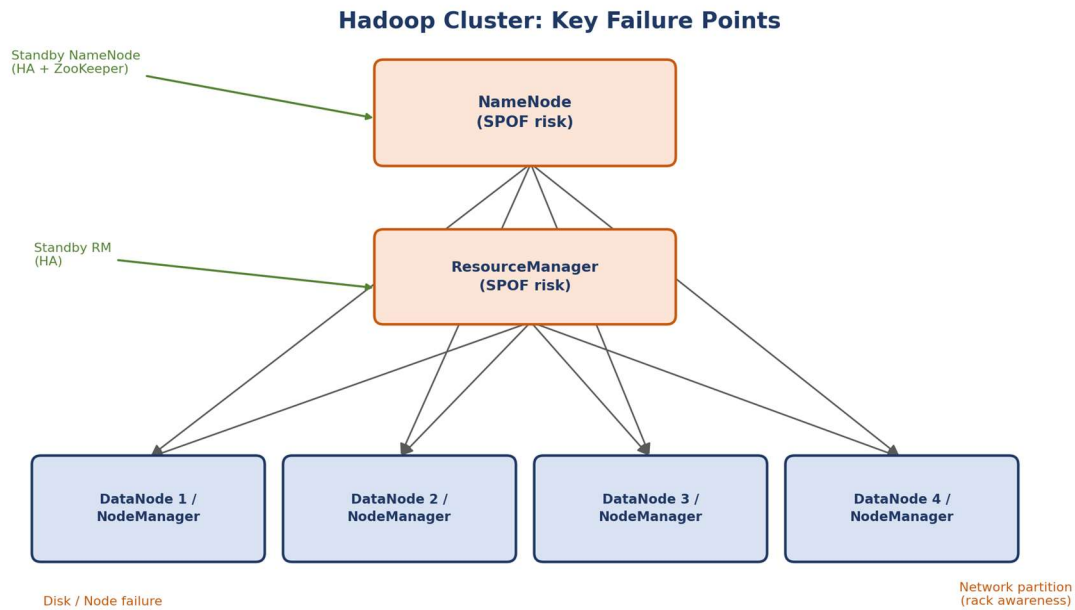


Figure 7: Key failure points across a Hadoop cluster — master services, worker nodes, and network paths.

Failure Point	Risk	Mitigation
NameNode	Single point of failure for HDFS metadata; cluster becomes unusable if it goes down	Deploy HDFS High Availability (Active/Standby NameNode) with automatic failover via ZooKeeper (ZKFC); take regular fsimage/edit-log checkpoints
ResourceManager	New job scheduling stops cluster-wide if it fails	Enable YARN ResourceManager HA (Active/Standby) coordinated through ZooKeeper
DataNode / disk failure	Loss of block replicas stored on that node	Rely on block replication (default factor 3); NameNode automatically re-replicates under-replicated blocks to healthy nodes
Network partition / rack failure	Nodes on an isolated rack become unreachable	Rack-aware replica placement ensures replicas exist on more than one rack; redundant network paths/switches

Task / container failure	An individual map or reduce task crashes	YARN/MRAppMaster automatically re-schedules failed tasks on healthy nodes, often using speculative execution for slow tasks
Data skew / hot nodes	Uneven key distribution overloads a few reducers or nodes	Custom partitioners, salting skewed keys, and combiners to balance load before the shuffle
Configuration/human error	Misconfiguration causes cascading failures	Infrastructure-as-code, staged rollouts, monitoring/alerting (e.g., Ambari, Cloudera Manager, Prometheus)

Operationally, this is backed by continuous monitoring (heartbeats, block reports, resource metrics), automated alerting, and periodic disaster-recovery testing so that failure handling isn't only theoretical but verified to work under real conditions.

Q9. How would you design a secure data ingestion pipeline for a large organization?

A secure ingestion pipeline needs to protect data at every stage — in transit, during processing, and at rest — while maintaining strong identity controls, auditability, and governed access. I would design it in layers:

- **Perimeter and transport security:** All source connections use TLS/SSL in transit; ingestion sits behind a gateway that terminates connections, applies rate limiting, and rejects malformed or unauthenticated traffic.
- **Authentication and authorization:** Every producer and consumer authenticates via a centralized identity provider — Kerberos for the Hadoop ecosystem, or enterprise IAM/SSO for external services — with role-based access control (RBAC) so systems and users only get the minimum access they need.
- **Ingestion layer:** Tools like Apache NiFi or Kafka handle intake with built-in provenance tracking, back-pressure, and the ability to route data into different pipelines based on sensitivity classification.
- **Validation and data protection:** Incoming records are schema-validated and sanitized; sensitive fields (PII, financial data) are masked, tokenized, or field-level encrypted before they ever reach long-term storage.
- **Encrypted storage:** Data at rest is stored in HDFS/data lake with transparent encryption (e.g., HDFS Transparent Data Encryption zones) and managed encryption keys via a dedicated key management service (KMS).
- **Audit logging and monitoring:** Every ingestion event, access attempt, and transformation is logged to a centralized audit trail and fed into a SIEM for real-time anomaly detection and compliance reporting.
- **Governance:** A data catalog with lineage tracking and RBAC/ABAC-based access policies ensures that even after ingestion, data can only be queried by authorized roles, satisfying regulatory requirements (e.g., GDPR, HIPAA).

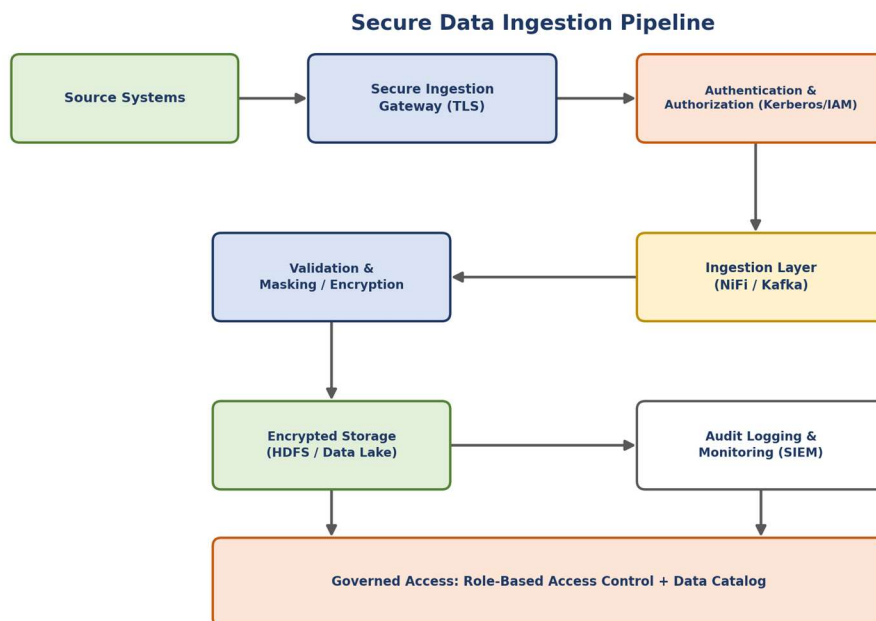


Figure 8: A secure ingestion pipeline layers transport security, identity, validation/encryption, and governed, audited access.

This layered ('defense in depth') approach ensures that a weakness at any single layer — a compromised credential, a misconfigured connector, an unencrypted field — doesn't by itself expose the organization's data.

Q10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mapping that back to the system:

- **The 'team of employees':** A cluster of many standard, inexpensive computers instead of one very expensive machine — this is what makes the system affordable to scale.
- **The 'backup copies':** Data replication — so hardware failure (which happens routinely at this scale) never means losing information or stopping work.
- **The 'site manager':** The system component that tracks where every piece of data lives and keeps everything organized (HDFS's NameNode).
- **The 'supervisor allocating desks':** The scheduler that shares out computing resources fairly across all the jobs running at once (YARN).
- **Splitting the paperwork:** Big jobs are automatically broken into small pieces that run in parallel, so processing that would take days on one machine can finish in a fraction of the time (MapReduce/Spark).

For a manager, the business takeaway is: this architecture lets us process far larger volumes of data, far faster and more cheaply, than a traditional single-server system — and it keeps running reliably even when individual machines fail, which is inevitable at this scale.